plan

Plan: LLMsVerifier Integration into HelixTranslate — Deep Analysis & Implementation Plan

Overview

Create an in-depth, enterprise-grade integration plan for LLMsVerifier as the single source of truth for model provisioning in HelixTranslate. Three repositories must be analyzed comprehensively.

Stage 1 — Repository Discovery & Deep Analysis

Goal: Clone and analyze all three repositories to understand architecture, code structure, APIs, configuration, testing, and documentation.

Sub-tasks:

1.1 Clone HelixTranslate repository and analyze:

- Project structure, architecture, entry points
- Model management & provider integration
- Configuration system (config files, .env support)
- Existing test suite & Challenges
- Documentation (CLAUDE.MD, AGENTS.MD, README, etc.)
- All supported providers, models, MCPs, LSPs, ACPs, Embeddings, RAGs, Skills, Plugins

1.2 Clone LLMsVerifier repository and analyze:

- Project structure, architecture, entry points
- Validation, verification, scoring mechanisms
- API surface (classes, methods, exports)
- Configuration requirements, API keys needed
- Test suite
- Documentation

1.3 Clone HelixAgent repository and analyze:

- How LLMsVerifier is already integrated here (reference implementation)
- Configuration pattern, initialization pattern
- Model consumption pattern from LLMsVerifier
- All providers, MCPs, LSPs, ACPs, Embeddings, RAGs, Skills, Plugins usage
- Test suite & Challenges structure
- Documentation (CLAUDE.MD, AGENTS.MD)

Parallel Execution:

- Agent 1: HelixTranslate deep analysis
- Agent 2: LLMsVerifier deep analysis
- Agent 3: HelixAgent deep analysis (reference integration)

Stage 2 — Gap Analysis & Integration Design

Goal: Compare and identify gaps, design the integration architecture.

Sub-tasks:

2.1 Model Management Gap Analysis

- How HelixTranslate currently gets models vs. LLMsVerifier's model provision
- What changes needed for single-source-of-truth pattern
- Provider abstraction alignment

2.2 Configuration Integration Design

- How to route LLMsVerifier API keys through HelixTranslate config/.env
- Configuration schema design
- Backward compatibility considerations

2.3 Full Capability Integration Design

- MCPs, LSPs, ACPs, Embeddings, RAGs, Skills, Plugins integration plan
- How each capability flows from LLMsVerifier → HelixTranslate

2.4 UX Design for Enterprise Translation Tool

- Model selection UX (validated models only)
- Provider management UX
- Error handling, fallback strategies
- Multi-model translation workflows

Stage 3 — Implementation Plan Writing

Goal: Write the comprehensive implementation plan document.

Sub-tasks:

3.1 Phase-based Implementation Plan

- Phase 1: Foundation & Configuration
- Phase 2: Core LLMsVerifier Integration
- Phase 3: Full Capability Integration (MCPs, LSPs, ACPs, etc.)
- Phase 4: UX & Enterprise Features
- Phase 5: Testing & Quality Assurance
- Phase 6: Documentation & Constitution Updates

3.2 Task & Sub-task Breakdown

- Fine-grained tasks with codebase references
- Exact files, functions, lines to modify
- New files to create

Stage 4 — Testing Strategy & Anti-Bluff Framework

Goal: Design comprehensive testing strategy with anti-bluff guarantees.

Sub-tasks:

4.1 Test Coverage Plan

- Unit tests, integration tests, e2e tests
- Challenge-based validation
- Model validation testing
- Provider integration testing

4.2 Anti-Bluff Testing Framework

- Constitution/CLAUDE.MD/AGENTS.MD requirements
- Real feature validation (not just mock tests)
- End-to-end workflow verification
- Submodule Constitution propagation

Stage 5 — Final Assembly & Document Production

Goal: Produce final comprehensive document in proper format.

Sub-tasks:

5.1 Assemble all analysis into final document 5.2 Convert to DOCX format using docx skill 5.3 Deliver final artifact

Skills Used:

- Stage 1-4: deep-research-swarm (multi-agent parallel analysis)

- Stage 5: report-writing + docx